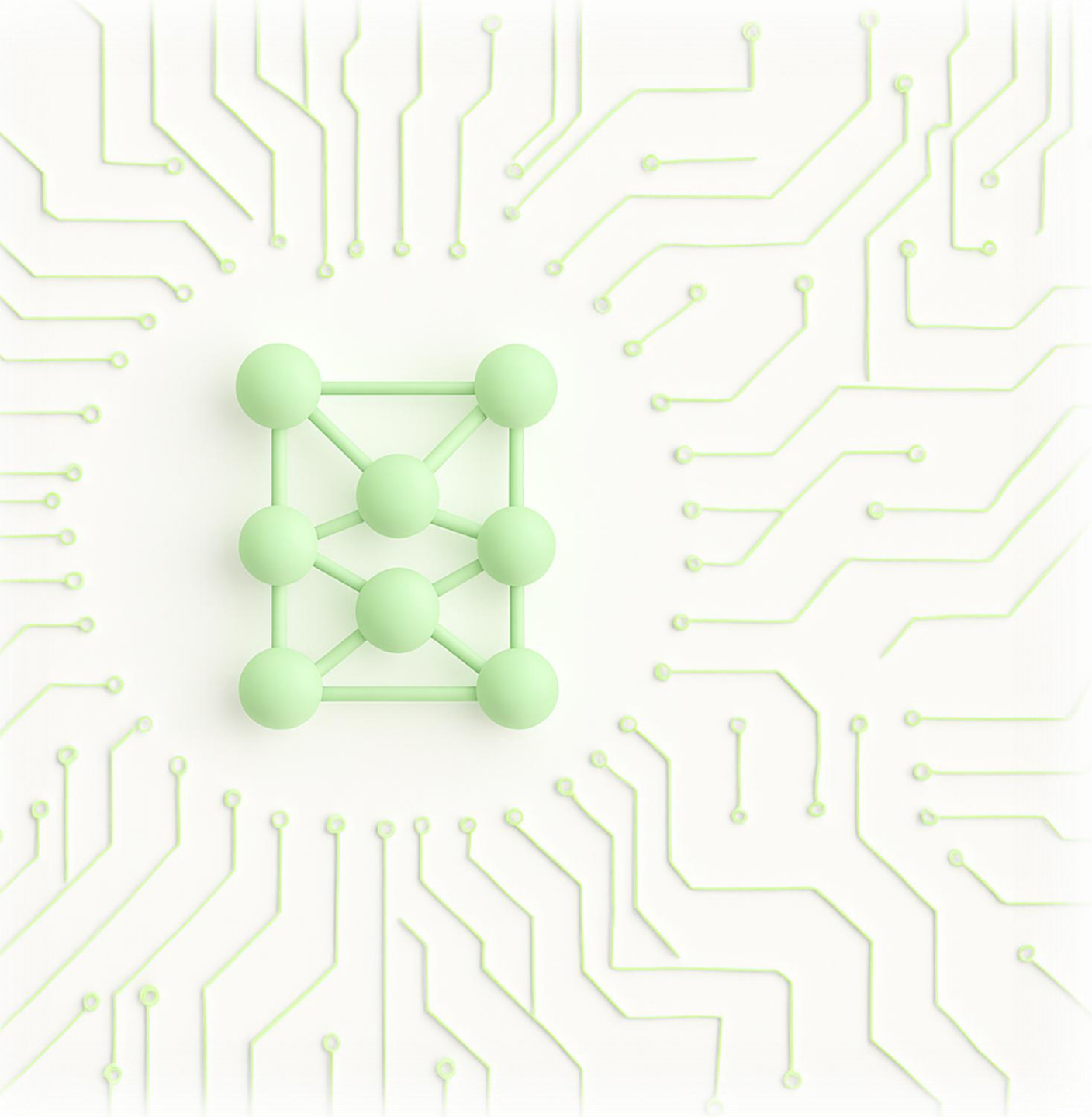




Deep Learning

Lakshmi Narasimhan
IIT Madras

Outline

1. Recap of Machine learning Fundamentals

- Linear algebra and probability
- Supervised learning: Classification and Regression
- Loss and errors
- Model selection: over/under fitting, Bias-Variance tradeoff

Outline

2. Multilayer Perceptron

- Function approximation theorems
- Feedforward neural networks
- Activation functions
- Width, depth and expressivity

Outline

3. Training neural networks

- Backpropagation
- Computational graph
- Optimization: Stochastic gradient descent and variants
- Avoiding overfitting: regularization, dropout, data augmentation
- Gradient instability, skip connection, initializations, early stop
- Normalization: input, layer, batch
- Hyperparameter tuning, curriculum training, transfer learning

Outline

4. Convolutional neural networks

- Multi-channel inputs
- Convolution and pooling
- Padding, striding, flattening
- Example architectures: AlexNet, GoogleNet, VGGNet, ResNet

Outline

5. Recurrent neural networks

- Recursive neural networks
- Backpropagation through time
- Neural nets with memory: LSTM and GRU
- Sequence learning and bidirectional LSTM
- Optimizing RNN: Teacher forcing, leaky units, skipping through time

Outline

6. Attention Mechanism

- Self attention and multi-head attention
- Query-key-value
- Attention pooling and scoring
- Embedding and encoding
- Transformer architecture

Outline

7. Autoencoders

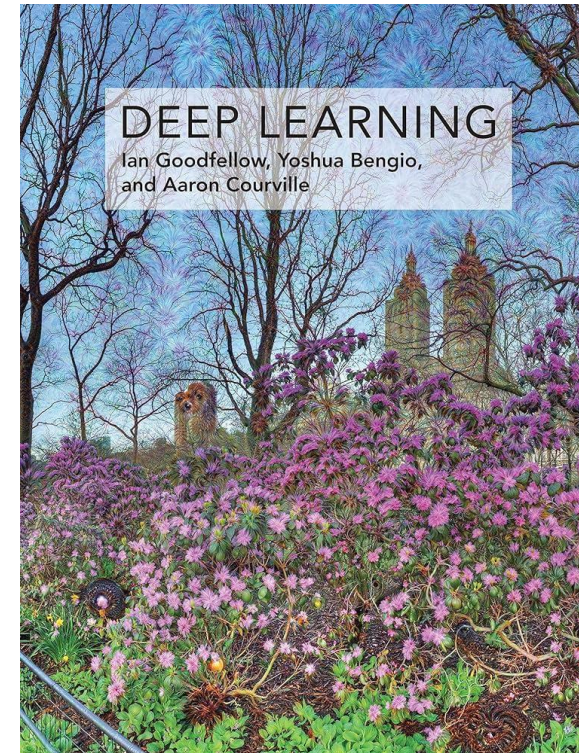
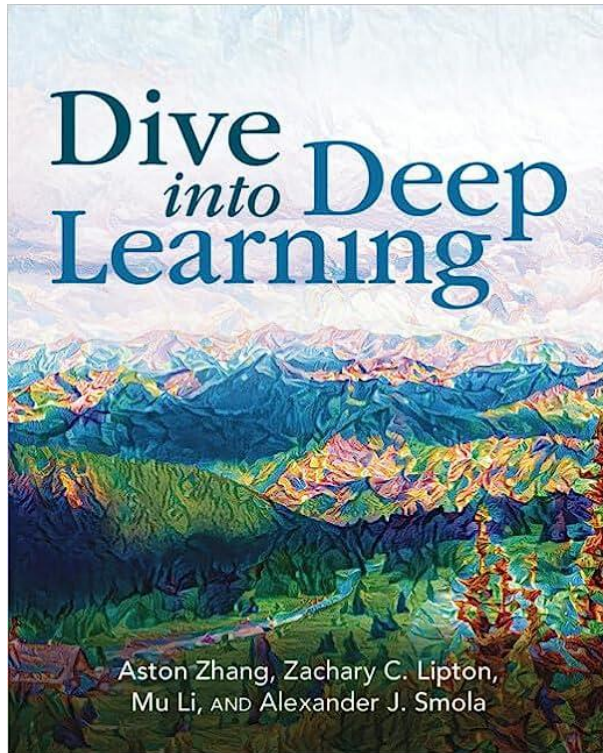
- Bottleneck architecture for encoder and decoders
- PCA/sparse decomposition with autoencoders
- Contractive and denoising autoencoders
- Variational autoencoder

Outline

8. State-of-the-art

- Graph neural networks
- Copresheaf Topological neural networks
- Physics informed neural networks
- Kolmogorov Arnold neural networks
- Diffusion and generative models

Reference materials



Logistics

- Lectures: 5.30 PM to 7 PM on Mon & Thu (Jan 5th to Apr 10th)
- Tutorials: On alternate weeks
- Assignments: One per two weeks
 - Theory: 50%
 - Programming: 50%
- Head TAs
 - Manoj Kumar (da24s018@smail.iitm.ac.in)
 - Yuvaram Singh (da24s015@smail.iitm.ac.in)

Grading

- 25% - Mid term, 22 Feb
- 25% - End term, 26 Apr
- 20% - Assignments (4 best out of 6)
- 30% - Project
 - Finalize topic by Feb 28
 - Deadline: April 22
 - Submit: Report + codes

Sample Project Ideas

- Sentence Sentiment Classification

- <https://www.kaggle.com/datasets/jp797498e/twitter-entity-sentiment-analysis>
- <https://www.kaggle.com/datasets/kazanov/sentiment140>

- Image Super Resolution

- <https://www.kaggle.com/datasets/adityachandrasekhar/image-super-resolution>
- <https://www.kaggle.com/datasets/jesucristo/super-resolution-benchmarks>

- GenAI with Variational Autoencoders

- <https://www.kaggle.com/code/basu369victor/generate-music-with-variational-autoencoder>
- <https://www.kaggle.com/code/averkij/variational-autoencoder-and-faces-generation>

- Product Recommender System

- <https://www.kaggle.com/datasets/shivamb/fashion-clothing-products-catalog>
- <https://www.kaggle.com/datasets/lokeshparab/amazon-products-dataset>

Project Report Format

- Introduction (1 mark)
- Background and state-of-the-art (1 mark)
- Problem Statement (4 marks)
- Dataset Description (2 marks)
- Solution/Methodology/Neural Net Architecture (4 marks)
- Theoretical Analysis (2 marks)
- Data Preprocessing and Training Methodology (3 marks)
- Experimental Results (5 marks for plots & description + 5 marks for attached code)
- Inferences/Insights Obtained/Difficulties faced (3 marks)
- Template - <https://www.overleaf.com/read/cbbbwmpqctbv#397b3a>

Linear algebra and probability

Vectorspaces

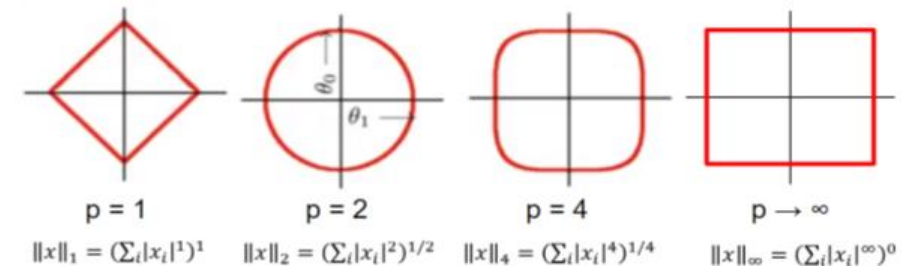
- A set $V : \forall \mathbf{x}, \mathbf{y} \in V, a\mathbf{x} + b\mathbf{y} \in V$, where a and b are any scalars
 - Closed under addition and scalar multiplication
- **Subspace**: a subset of V that is closed under addition and scalar multiplication
- Any element of this vectorspace is a **vector**
 - For finite-dimensional vectorspaces (e.g., $\mathbb{R}^n$): $n \times 1$ array of numbers
 - In our study, most data samples will be represented by a vector

Orthogonality & Norms

- An inner product on $\mathbb{R}^n$: $\mathbf{x}^T \mathbf{y}$
 - Measure of component of $\mathbf{x}$ on $\mathbf{y}$ (and vice versa)
 - **Orthogonal**: $\mathbf{x}^T \mathbf{y} = 0$ (perpendicular vectors)
 - $\mathbf{x}^T \mathbf{y} > 0$: $\mathbf{x}$ and $\mathbf{y}$ have components along the same direction
 - $\mathbf{x}^T \mathbf{y} < 0$: $\mathbf{x}$ and $\mathbf{y}$ have components along opposite directions

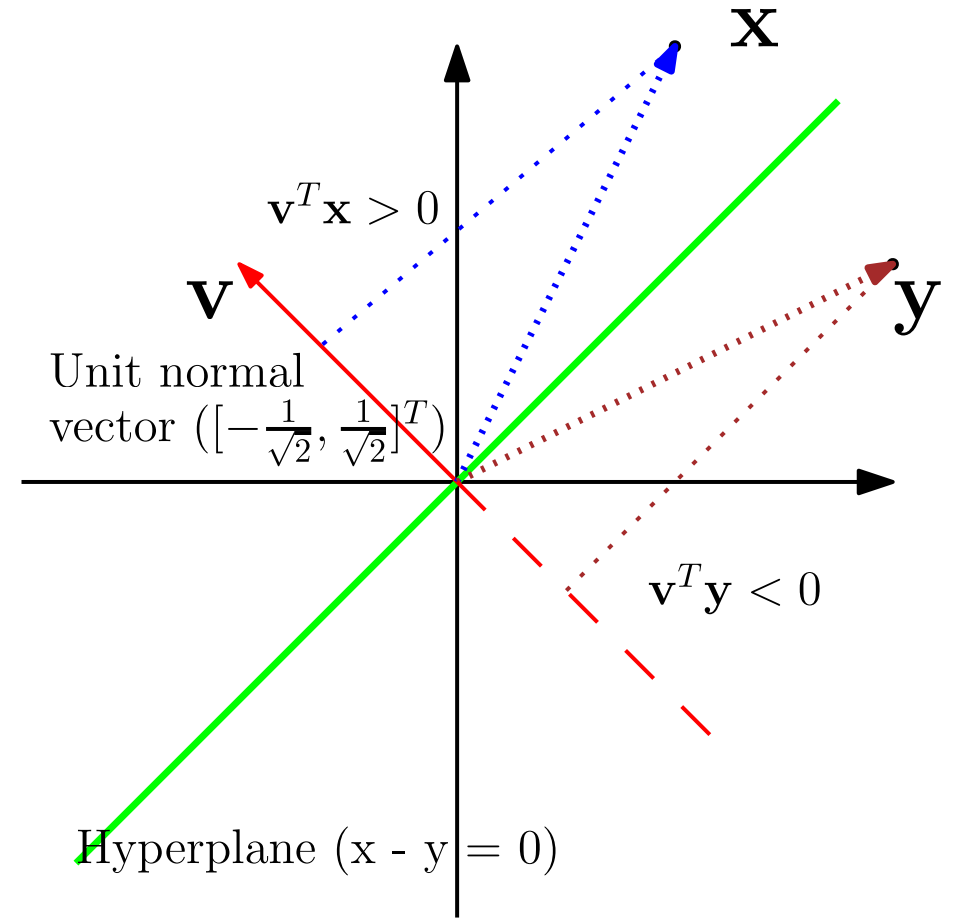
- **p - norm** ($p \geq 1$) : $\|\mathbf{x}\|_p := \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$

- **Euclidean norm**: $\|\mathbf{x}\|^2 = \mathbf{x}^T \mathbf{x}$



Hyperplanes

- **Linear hyperplane**
 - An $n - 1$ dimensional subspace of $\mathbb{R}^n$
 - Passes through the origin
 - Divides $\mathbb{R}^n$ into two halves
 - Represented by a normal vector
- **Normal vector:**
 - A vector orthogonal to all vectors on the hyperplane
 - Distance of $\mathbf{x}$ from hyperplane = $\mathbf{x}^T \mathbf{v} / \|\mathbf{v}\|$



Singular Value Decomposition

- Singular value decomposition : $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
 - Exists for any matrix
 - $\mathbf{U}, \mathbf{V}$ are orthogonal: left and right **singular vectors**
 - $\mathbf{\Sigma}$ is PSD diagonal: The diagonal elements are **singular values**
 - The diagonal elements of $\mathbf{\Sigma}$ are in ascending order : $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$
- **Pseudoinverse**: $\mathbf{V}\mathbf{\Sigma}_r^{-1}\mathbf{U}^T$, where r is the rank of $\mathbf{M}$
- **Matrix norm**: $\|A\|_p = \sup_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p}$ $\|A\|_2 = \sigma_{\max}(A)$

Random Variable

- Almost all data collected in real life are affected by noise
 - This measurement is an outcome of the data collection experiment
 - The outcome may change every time the experiment is performed
 - However, certain *statistical* properties of these outcomes are deterministic
- **Probability space:** (Ω, F, P) – quantifying randomness of measurements
 - Ω : Sample space; F : Event space – all possible subsets of Ω
 - P : a measure for all elements of F – e.g., proportion of occurrence of any $u \in F$
- Random variable: a mapping from Ω to $\mathbb{R}$ (real numbers)

PMF vs PDF

Discrete random variables	Continuous random variables
Ω is countable all $w \in \Omega$ has $0 \leq P(w) \leq 1$	Ω is uncountable $u \in F$ has $0 \leq P(u) \leq 1$, $w \in \Omega$ has $P(w) = 0$
$X(w)$ is a real-discrete-valued r.v. $P(w)$ is its probability <i>mass</i>	$X(w)$ is a real-continuous-valued r.v. $P(w)$ is its probability <i>density</i>
Probability mass function (PMF) $P(X = x) = P(w)$ for $X(w) = x$	Probability density function (PDF) $P(X \in [a, b]) = \int_a^b P(x) dx = P(u)$, $u = \{X^{-1}(x), x \in [a, b]\}$
Law of total probability $\sum_i P(X = x_i) = P(\Omega) = 1$	Law of total probability $\int P(x) dx = P(\Omega) = 1$

Conditional Probability & Independence

- $P(X | Y)$: probability of occurrence of X when Y has occurred
 - The **sample space is reduced** by conditioning
 - Creates a new probability space and a valid PMF/PDF – $P(X = x | Y)$
 - Probability can increase or decrease upon conditioning
 - $P(X | Y) = P(X, Y) / P(Y)$
 - $P(X | Y = y)$ is a function of x , but $P(X | Y)$ is a function of both x and y
- **Independent** r.v.s: X and Y do not influence each other
 - $P(X, Y) = P(X) P(Y) \implies P(X | Y) = P(X)$

Bayes Theorem

- $P(X | Y) = P(Y | X) P(X) / P(Y)$
 - Given that Y is observed and a dependent hidden variable X exists, we can infer the probability of the hidden variable
- $P(X)$: Prior, statistics known before any observation
- $P(X | Y)$: Posterior, computed indirectly after observing Y
 - $P(X | Y = y)$ is a valid PMF/PDF; $P(X | Y = y) \propto P(Y = y | X) P(X)$
- $P(Y | X)$: Likelihood, computed based on the outcome of Y
 - $P(Y = y | X)$ is not a valid PMF/PDF
 - Note that the probability space itself changes for each value of X

Gaussian Distribution

- Gaussian $X \sim \mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{\sigma^2}}$
 - μ, σ : Only two parameters describe the entire PDF
- **Multivariate Gaussian:** $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{(2\pi)^n \det \boldsymbol{\Sigma}}} e^{-(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x}-\boldsymbol{\mu})}$
 - $\mathbf{x} = [X_1, \dots, X_n]^T$ has n Gaussian r.v.'s that are also **jointly** Gaussian
 - $\boldsymbol{\mu} = \mathbb{E}\{\mathbf{x}\}$ is the mean vector, $\boldsymbol{\Sigma} = \mathbb{E}\{(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^T\}$ is a PSD covariance matrix
 - If $\mathbf{z} = \boldsymbol{\Sigma}^{-1/2} \mathbf{x}$, then the elements of $\mathbf{z}$ are independent Gaussian r.v.
 - Precision parameter: $\boldsymbol{\beta} = \boldsymbol{\Sigma}^{-1}$

KL Divergence

- **Maximum entropy distributions** (priors with least bias)
 - Uniform for discrete r.v.
 - Gaussian for continuous r.v.
- **Relative entropy**: $D(P \parallel Q) = \mathbb{E}_P\{\log P/Q\} \geq 0$
 - Measure of how distant is the distribution Q is different from P
- **Cross entropy**: $H(P; Q) = \mathbb{E}_P\{-\log Q\} \geq 0$ (for discrete r.v.)
 - $H(P; Q) = D(P \parallel Q) + H(P)$
 - Minimizing CE is equivalent to minimizing KLD

Classical vs Bayesian

Bayesian	Classical
Parameter to be estimated is a random variable	Parameter to be estimated is an unknown deterministic quantity
A prior distribution on the parameters is used	No prior (all values are equally likely - special case of Bayesian)
Posterior probability is optimized	Likelihood is optimized
Poor performance when prior is incorrect	Poor performance whenever the parameter is not uniformly distributed

Taylor Expansion

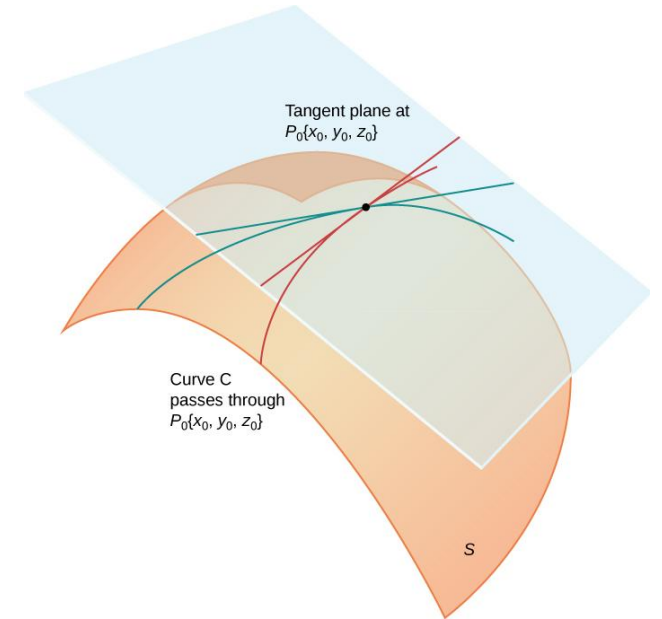
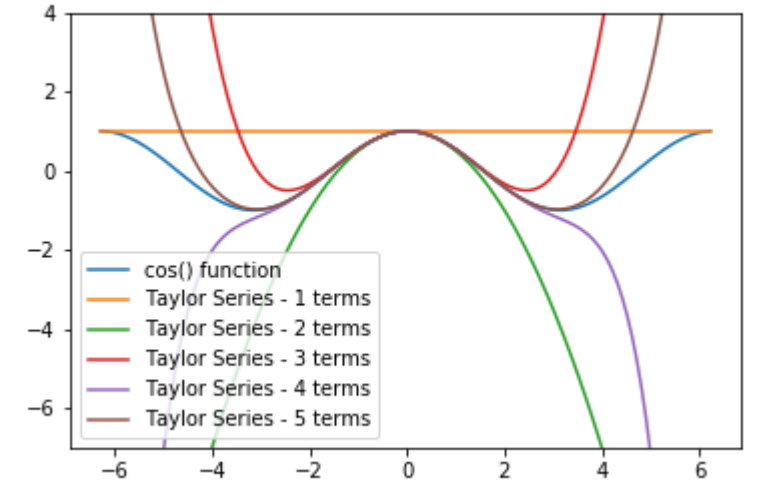
- Polynomial approximation

$$T(\mathbf{x}) = f(\mathbf{a}) + (\mathbf{x} - \mathbf{a})^\top Df(\mathbf{a}) + \frac{1}{2!}(\mathbf{x} - \mathbf{a})^\top \{D^2 f(\mathbf{a})\} (\mathbf{x} - \mathbf{a}) + \dots$$

- Gradient & Hessian

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$

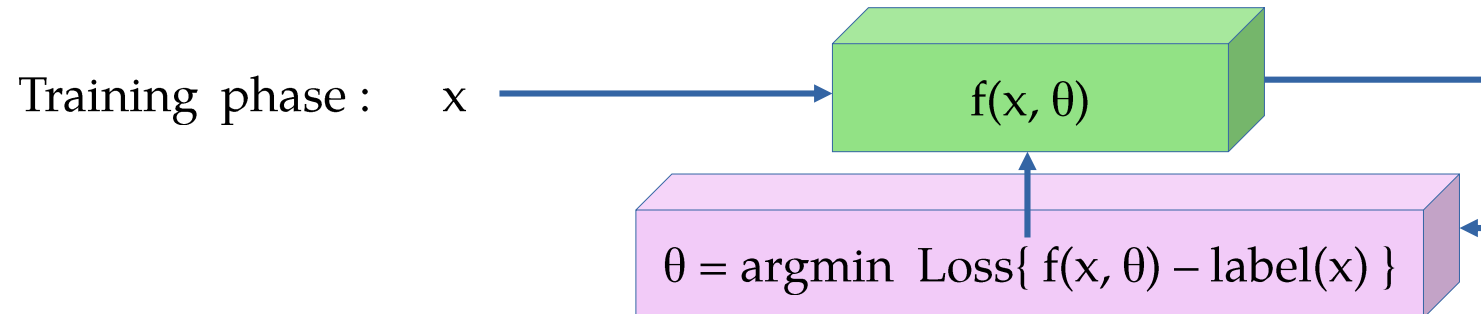
$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \dots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$



Machine Learning Fundamentals

Supervised/Unsupervised Learning

Supervised Learning	Unsupervised Learning
Training + Testing (inference) phase	Only inference phase
Requires labeled data for training	All data are unlabeled
Examples: Classification & estimation	Clustering, PCA, pattern finding

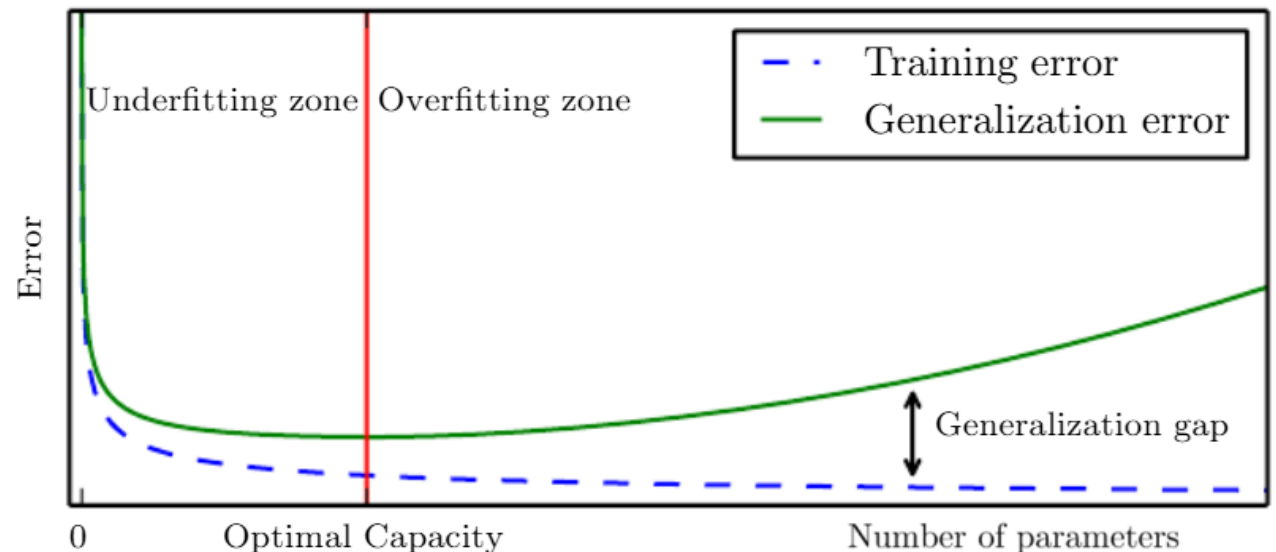


Loss Functions

- How to find the best parameters while training a supervised learning model?
 - Minimize the error between model output for input x_i and the label of x_i
- Loss function: A quantification of the error obtained during parameter optimization $\theta = \operatorname{argmin} \operatorname{Loss}\{f(x, \theta) - \operatorname{label}(x)\}$
 - **L0** norm (sparsity-promoting), 0-1 loss: $\|y - y_{\text{true}}\|_0$
 - **L1** norm, absolute error: $\|y - y_{\text{true}}\|_1$
 - **Squared L2** norm, Euclidean distance: $\|y - y_{\text{true}}\|_2^2$
 - Mixed norms: linear combination of different p -norms ($p \geq 1$) – **convex losses**
 - Cross entropy, F-score, etc.

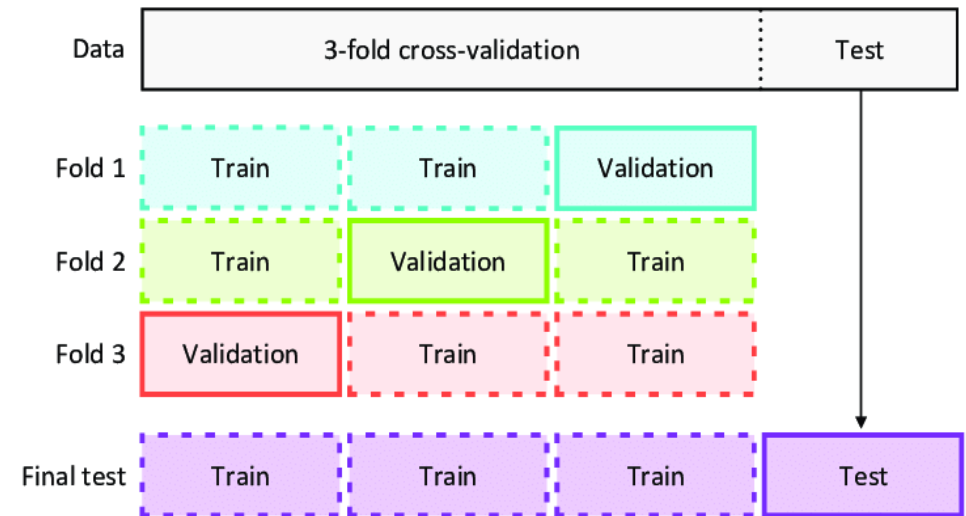
Training/Generalization Error

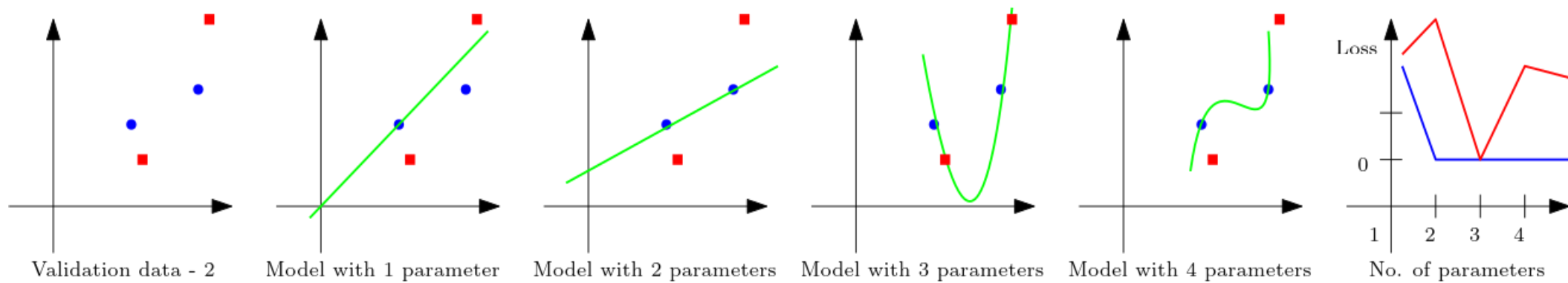
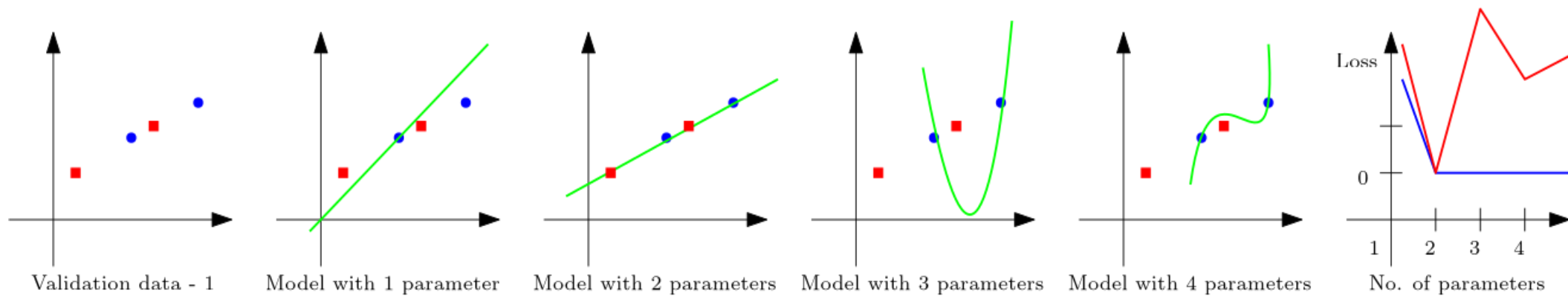
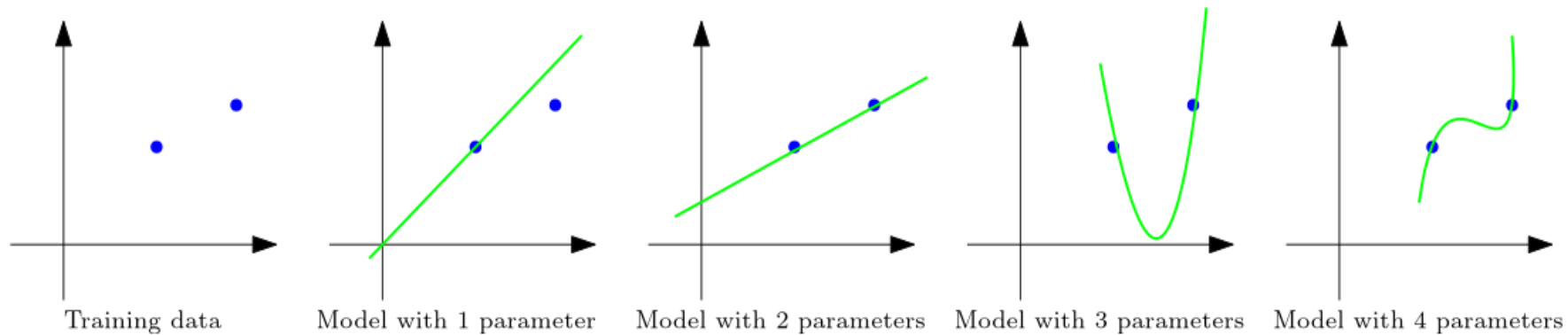
- Increasing number of parameters is not always good
 - Training error may go down, but testing (generalization) error may be high
- There is an optimal number of parameters for a given kind of data
 - VC dimension
- Underfitting: Less than optimal
- Overfitting: More than optimal
 - How to identify overfitting?



Validation Error & Overfitting

- Split available labeled data to training + validation sets
 - Sequentially minimize both training and validation errors
 - Use validation set to identify overfitting
- Validation error: The error measured with validation set – An indicator of generalization error
- **Cross validation:** When very few labeled data is available
 - Use different training+validation split in each iteration of training





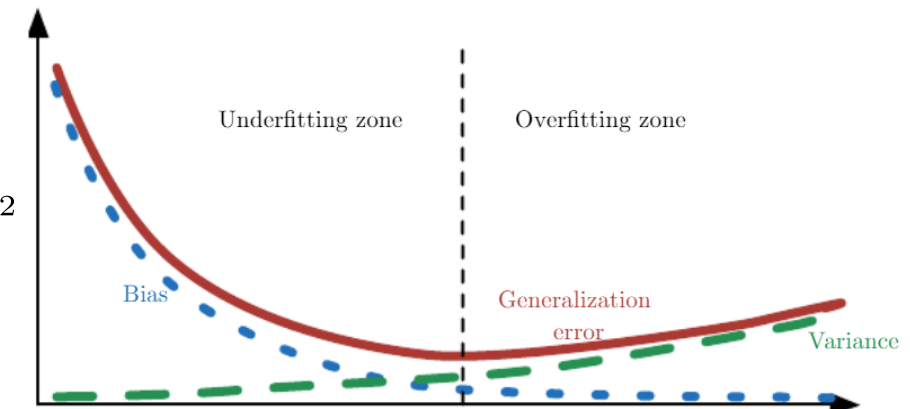
No Free-Lunch Theorem

- Training dataset may not contain all possible samples of input
 - How would the unseen data affect the learned statistics?
- **No free-lunch theorem**: As the unseen data can have different statistics, there can exist a dataset where any model (optimally trained with a training dataset) can perform poorly
 - So, will all models perform equally bad on average?
 - Tradeoff between **robustness** and **accuracy**
- **Regularization**: If some statistics of the entire data is known, use that information in addition to training data to optimize the model

Bias-Variance & Model-fitting

- Bias: $\mathbb{E}\{y - y_{\text{true}}\}$ – average deviation from true value
- Variance: $\text{Var}\{y\}$ – average deviation of inference
- Is lower bias or variance always good?

$$\begin{aligned}\mathbb{E}((y - \hat{y})^2) &= \mathbb{E}(y^2) + \mathbb{E}(\hat{y}^2) - 2\mathbb{E}(\hat{y})\mathbb{E}(y) \\ &= \mathbb{E}(y^2) - \mathbb{E}(y)^2 + \mathbb{E}(\hat{y}^2) - \mathbb{E}(\hat{y})^2 - 2\mathbb{E}(\hat{y})\mathbb{E}(y) + \mathbb{E}(y)^2 + \mathbb{E}(\hat{y})^2 \\ &= \text{Var}(y) + \text{Var}(\hat{y}) + \text{Bias}^2\end{aligned}$$



- Overfitting : High model variance and low MSE in training
- Underfitting : Low model variance and high MSE in training

Bayesian Inference

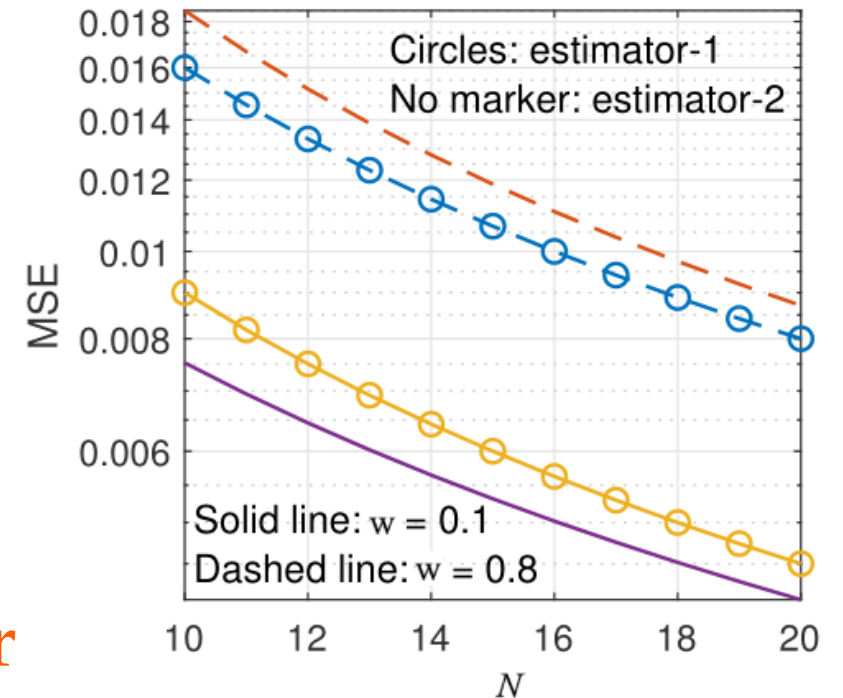
- Infer bias (p) of a coin from observations of its toss: $X_i \sim \text{Be}(p)$

$$\hat{p}_1 = \arg \max_p p^{\sum y_i} (1 - p)^{N - \sum y_i} = \frac{\sum y_i}{N}$$

$$\text{Var}(\hat{p}_1) = \frac{p(1 - p)}{N} \quad \text{Bias}(\hat{p}_1) = 0$$

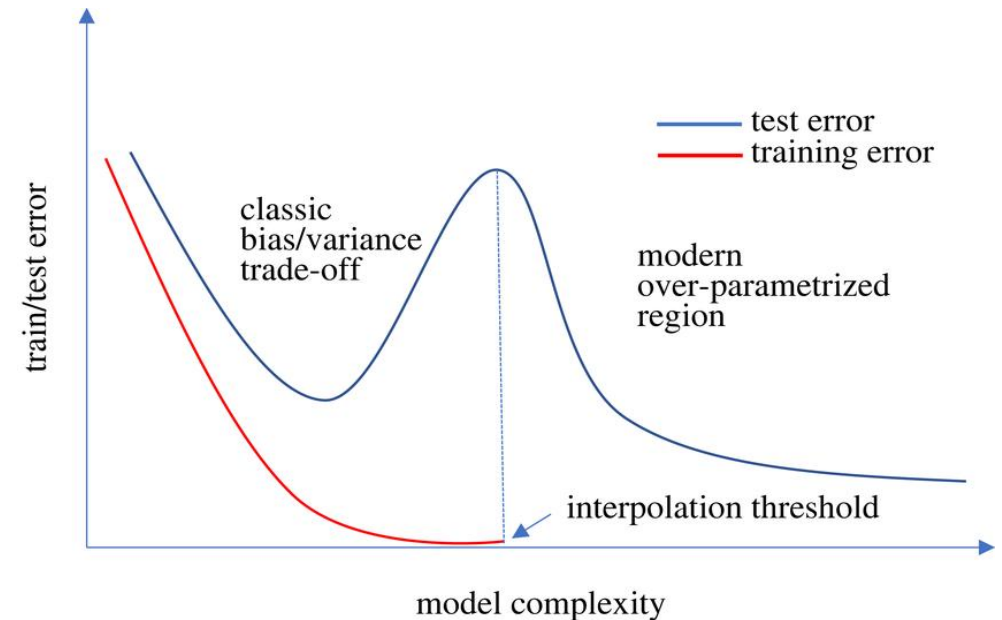
$$\hat{p}_3 = \arg \max_p p^{\sum y_i} (1 - p)^{N - \sum y_i} P(p) = \frac{\sum y_i}{N + 1}$$

- (Classical) No bias
- (Bayesian) Bias is already in-built in prior



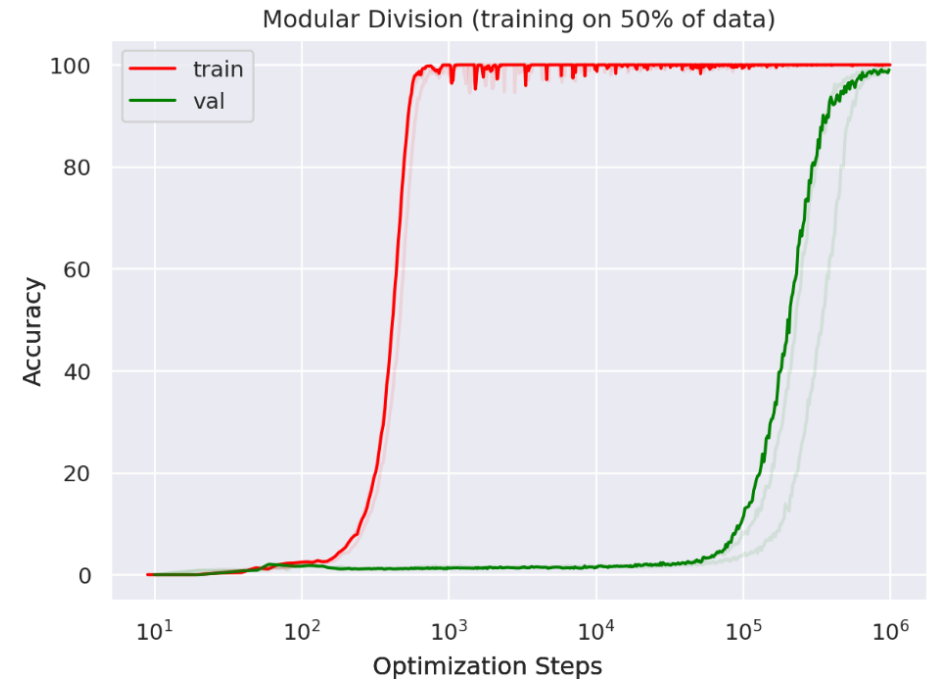
Double Descent

- Overfitting can be overcome
 - With deeper networks
- As the number of parameters (depth) increases, the overfitting is smoothened
 - Smoother interpolations and extrapolations
- Interpolation threshold
 - Where the number of parameters equals the number of data points
 - R. Shaeffer et al, *Double Descent Demystified: Identifying, Interpreting & Ablating the Sources of a Deep Learning Puzzle*
 - Some times more training data can hurt performance!



Grokking

- Generalization of over-parameterized nets by **over training**
- Amount of over training reduces with decrease in dataset size
 - *Grokking Explained: A Statistical Phenomenon, 2025*
- **Weight decay** helps
 - Additional term in the loss function to push the weights closer to zero



Regression

- Model: $y_i = f(x_i) + \varepsilon$
- Find: $f(\cdot)$. Parameterized $f(x) = g(\mathbf{w}, x)$; linear regression: $g(\mathbf{w}, \mathbf{x}) = \mathbf{W}^T \mathbf{x}$
- Given (training data)
 - Labels: $\{y_i\}$
 - Inputs : $\{x_i\}$
- Predict (testing data)
 - Compute $\{y\}$ for unseen $\{x\}$

Classification

- Model: $x_{i,j} \in C_i$; $i = 1, \dots, K$ and $j = 1, \dots, N$
- Find: $f(.)$ s.t. $f(x_{i,j}) = C_i$
- Given (training data)
 - Labels: $\{y_i\}$ s.t. $y_i \in \{C_i\}$
 - Inputs : $\{x_{i,j}\}$
- Predict (testing data)
 - Compute $\{y\}$ for unseen $\{x\}$ s.t. $y \in \{C_i\}$

Classification

- Binary classification: $K = 2$, $C_1 = +1$ and $C_2 = -1$
- Posterior probability for binary classification is

$$\begin{aligned}\Pr(y_i = +1|x_i) &= \frac{\Pr(x_i|y_i = +1) \Pr(y_i = +1)}{\Pr(x_i|y_i = +1) \Pr(y_i = +1) + \Pr(x_i|y_i = -1) \Pr(y_i = -1)} \\ &= \frac{1}{1 + e^{-a}} = \sigma(a); \quad a = \ln \frac{\Pr(x_i|y_i = +1) \Pr(y_i = +1)}{\Pr(x_i|y_i = -1) \Pr(y_i = -1)}\end{aligned}$$

$a = \log$ posterior ratios

- The **sigmoid** function naturally arises here

Classification

- Posterior probability for K-class classification is

$$\begin{aligned}\Pr(y_i = C_k | x_i) &= \frac{\Pr(x_i | y_i = C_k) \Pr(y_i = C_k)}{\sum_j \Pr(x_i | y_i = C_j) \Pr(y_i = C_j)} \\ &= \frac{e^{a_k}}{\sum_j e^{a_j}} = \sigma_k(\mathbf{a}); \quad a_k = \ln \Pr(x_i | y_i = C_k) \Pr(y_i = C_k)\end{aligned}$$

- The **softmax** function naturally arises here

Classification on Gaussian Data

$$\Pr(\mathbf{x}_i | y_i = 1) = \frac{1}{(2\pi)^{M/2} \det(\mathbf{\Sigma})^{1/2}} e^{-\frac{(\mathbf{x}_i - \boldsymbol{\mu}_k)^T \mathbf{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_k)}{2}}$$

$$\Pr(y_i = +1 | \mathbf{x}_i) = \sigma(\mathbf{x}_i^T \mathbf{w} + w_0); \quad \mathbf{w} = \mathbf{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_{-1})$$

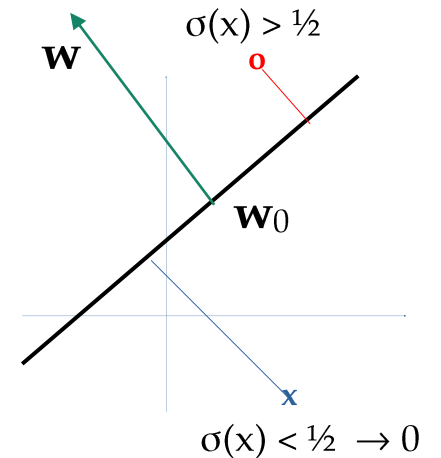
$$w_0 = -\frac{\boldsymbol{\mu}_1^T \mathbf{\Sigma}^{-1} \boldsymbol{\mu}_1}{2} + \frac{\boldsymbol{\mu}_{-1}^T \mathbf{\Sigma}^{-1} \boldsymbol{\mu}_{-1}}{2} + \ln \frac{\Pr(y_i = 1)}{\Pr(y_i = -1)}$$

$$\Pr(y_i = +1 | \mathbf{x}_i) = \sigma_k(\mathbf{x}_i^T \mathbf{W} + \mathbf{w}_0); \quad \mathbf{W} = \mathbf{\Sigma}^{-1}[\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K]$$

$$\mathbf{w}_0 = \left[-\frac{\boldsymbol{\mu}_1^T \mathbf{\Sigma}^{-1} \boldsymbol{\mu}_1}{2} + \ln \Pr(y_i = 1), \dots, -\frac{\boldsymbol{\mu}_K^T \mathbf{\Sigma}^{-1} \boldsymbol{\mu}_K}{2} + \ln \Pr(y_i = K) \right]$$

Classifying Linearly Separable Data

- Hyperplane projection: $(\mathbf{x} - \mathbf{w}_0)^T \mathbf{w} = \mathbf{x}^T \mathbf{w} - \mathbf{w}_0^T \mathbf{w} = \mathbf{x}^T \mathbf{w} - w_0$
 - $\mathbf{w}_0$ is the **shift of origin** and $\mathbf{w}$ is **normal to the hyperplane**
- Binary classification
 - When $\mathbf{x}$ is on the hyperplane, $\Pr(y = C_1) = \sigma(0) = 1/2$
 - Farther from the hyperplane $\sigma(+ve) \rightarrow 1$ or $\sigma(-ve) \rightarrow 0$
 - Sigmoid output gives the PMF of the **Bernoulli** r.v. $f(x)$
- Multi-class classification
 - When $\mathbf{x}$ is on the hyperplane, $\Pr(y = C_1) = \sigma_k(\mathbf{0}) = 1/K$
 - Farther from the k^{th} hyperplane $\sigma_k(+ve) \rightarrow 1$ or $\sigma_k(-ve) \rightarrow 0$
 - Softmax output gives the PMF of the **categorical** r.v. $f(x)$



Logistic Regression

- Let the classes be **one-hot encoded** (to represent a PMF)

$$\Pr(\mathbf{y}|\mathbf{x}, \mathbf{W}) = \prod_{k=1}^K \Pr(y = C_k|\mathbf{x}, \mathbf{W})^{y_k}; \quad \sum_{k=1}^K y_k = 1; \quad \sum_{k=1}^K \Pr(y = C_k|\mathbf{x}, \mathbf{W}) = 1$$

- To find $\mathbf{w}$: do **maximum likelihood regression**

$$\arg \min_{\mathbf{W}} -\ln \Pr(\mathbf{y}|\mathbf{x}, \mathbf{W}) = \arg \min_{\mathbf{W}} - \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \ln \sigma_k(\mathbf{x}_i^T \mathbf{W}) = \arg \min_{\mathbf{W}} \sum_i H(y_i; \sigma_k(\mathbf{x}_i^T \mathbf{W}))$$

- The optimal hyperplanes are obtained using gradient descent

$$\mathbf{w}_{j,t+1} = \mathbf{w}_{j,t} - \eta \nabla_{\mathbf{w}_j} \mathcal{L} = \mathbf{w}_{j,t} - \eta \sum_i (\sigma_j(\mathbf{x}_i^T \mathbf{W}_t) - y_{i,j}) \mathbf{x}_i$$